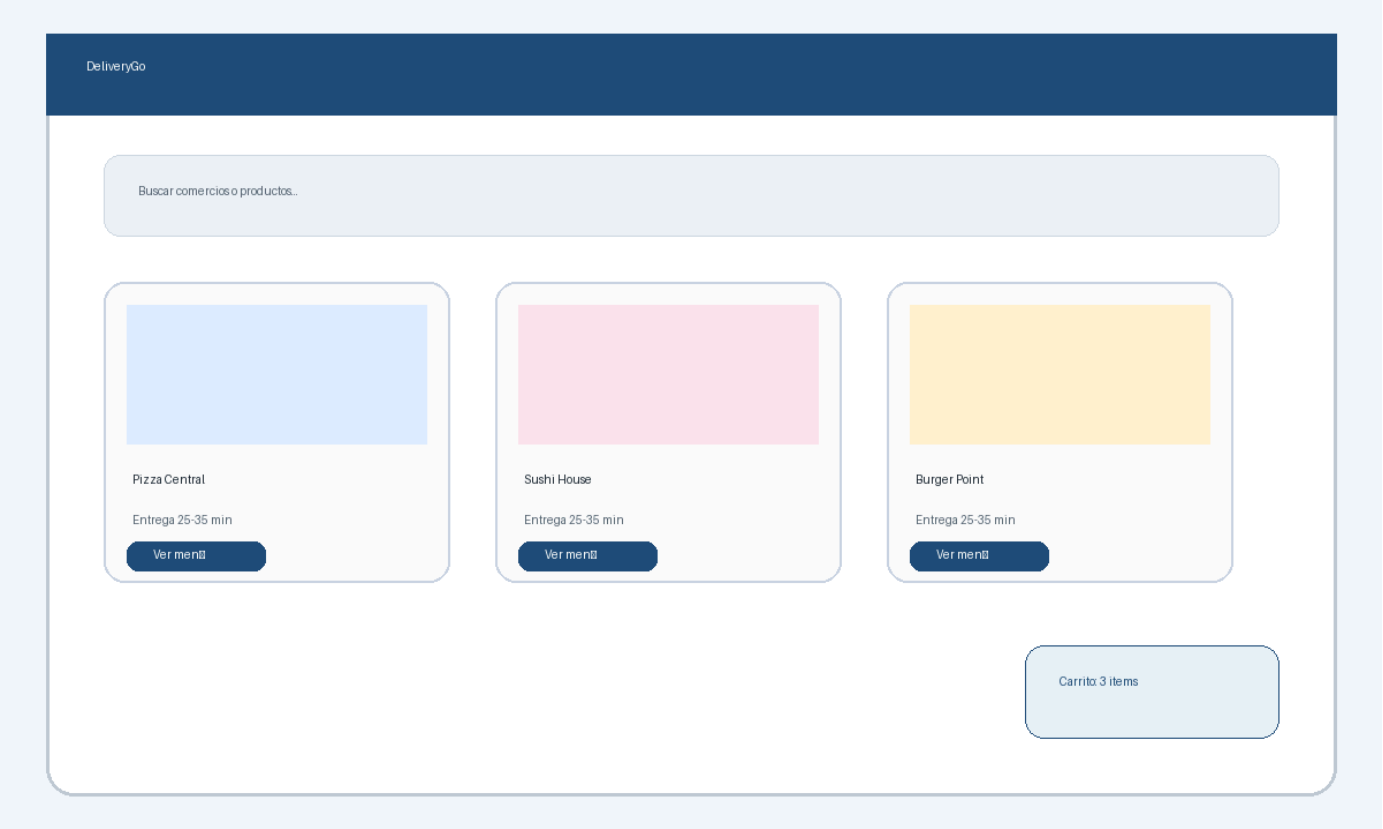
Trabajo Práctico Obligatorio

Ecosistema distribuido DeliveryGo

APLICACIONES INTERACTIVAS

Proyecto integrador para 33 estudiantes organizados en 8 equipos



Versión para estudiantes | Segundo cuatrimestre 2026

1. Introducción y contexto de DeliveryGo

DeliveryGo es una plataforma argentina ficticia de pedidos y entregas a domicilio. El caso fue diseñado para que todos los equipos desarrollen aplicaciones independientes, pero integradas en un ecosistema común mediante APIs REST.

Escenario

La empresa comenzó como una aplicación monolítica y creció rápidamente. Actualmente opera con clientes, comercios, repartidores, personal administrativo y responsables de negocio. El sistema debe soportar el registro de usuarios, publicación de catálogos, creación de pedidos, pagos, asignación de entregas, notificaciones y análisis de información.

Ustedes conformarán la Software Factory contratada para diseñar, construir, documentar e integrar la nueva plataforma.

Propósito formativo

- Aplicar arquitectura por capas y principios de diseño.
- Diseñar contratos entre aplicaciones.
- Persistir datos con tecnologías ORM.
- Construir clientes web y servicios backend.
- Integrar módulos desarrollados por equipos distintos.
- Defender decisiones técnicas con evidencia.

1. Introducción y contexto de DeliveryGo

Actores del ecosistema

Actor	Necesidad principal
Cliente	Buscar productos, comprar, pagar y seguir pedidos.
Comercio	Administrar catálogo, disponibilidad y preparación.
Repartidor	Aceptar entregas, informar ubicación y cerrar pedidos.
Administrador	Gestionar incidencias, auditoría y configuración.
Analista	Consultar métricas y tendencias del negocio.

Alcance general

- Aplicación web integradora.
- Ocho servicios o sistemas con límites funcionales claros.
- Persistencia independiente por módulo.
- Comunicación HTTP/REST con JSON.
- Documentación OpenAPI y diagramas C4/UML.
- Pruebas de integración y demostración final.

Fuera de alcance

- Dinero real o pasarelas bancarias reales.
- Geolocalización en tiempo real con mapas comerciales.
- Infraestructura productiva de alta disponibilidad.
- Reconocimiento biométrico o datos sensibles reales.

2. Problema de negocio y objetivos del proyecto

Problemas actuales

Área	Problema
Usuarios	Datos duplicados, direcciones inconsistentes y autenticación fragmentada.
Catálogo	Productos sin estandarización, precios desactualizados y disponibilidad incierta.
Pedidos	Estados inconsistentes, falta de trazabilidad y operaciones duplicadas.
Pagos	Confirmaciones manuales, ausencia de idempotencia y conciliación limitada.
Logística	Asignaciones tardías y poca visibilidad sobre las entregas.
Notificaciones	Mensajes repetidos y sin trazabilidad.
Analítica	Indicadores calculados de forma diferente por cada área.

Necesidad

DeliveryGo requiere reemplazar el sistema actual por un ecosistema modular. Cada dominio debe evolucionar de manera independiente, pero respetando contratos comunes y manteniendo una experiencia integrada para el usuario.

2. Problema de negocio y objetivos del proyecto

Objetivo general

Diseñar e implementar un ecosistema de aplicaciones interconectadas que permita completar el ciclo de un pedido de punta a punta, desde la autenticación hasta la entrega y el análisis posterior.

Objetivos específicos

1. Definir responsabilidades y límites de cada módulo.
2. Modelar entidades, reglas y estados del dominio.
3. Implementar servicios REST documentados.
4. Construir una interfaz web que consuma los servicios.
5. Resolver errores de integración y datos inconsistentes.
6. Gestionar el código con Git y revisiones por Pull Request.
7. Demostrar la solución integral y defenderla individualmente.

Criterio de éxito

El TPO se considera integrado cuando un usuario puede registrarse, seleccionar productos, crear y pagar un pedido, recibir la asignación de un repartidor, consultar su estado y recibir notificaciones; los datos del proceso deben estar disponibles para analítica.

3. Arquitectura del ecosistema

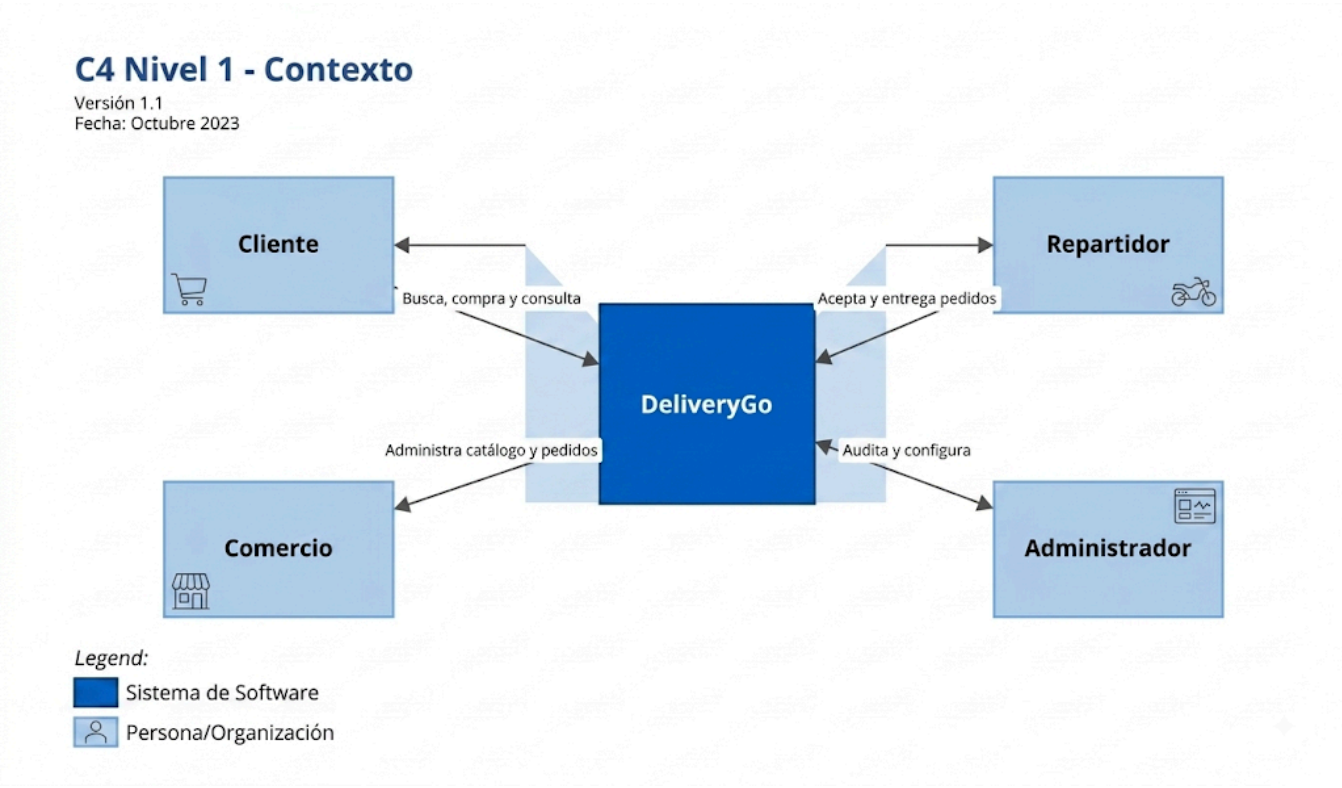


Figura 1. Contexto general de DeliveryGo.

El sistema centraliza la experiencia del cliente, comercio y repartidor, pero internamente distribuye responsabilidades entre varios servicios.

3. Arquitectura del ecosistema

C4 Nivel 2 - Contenedores

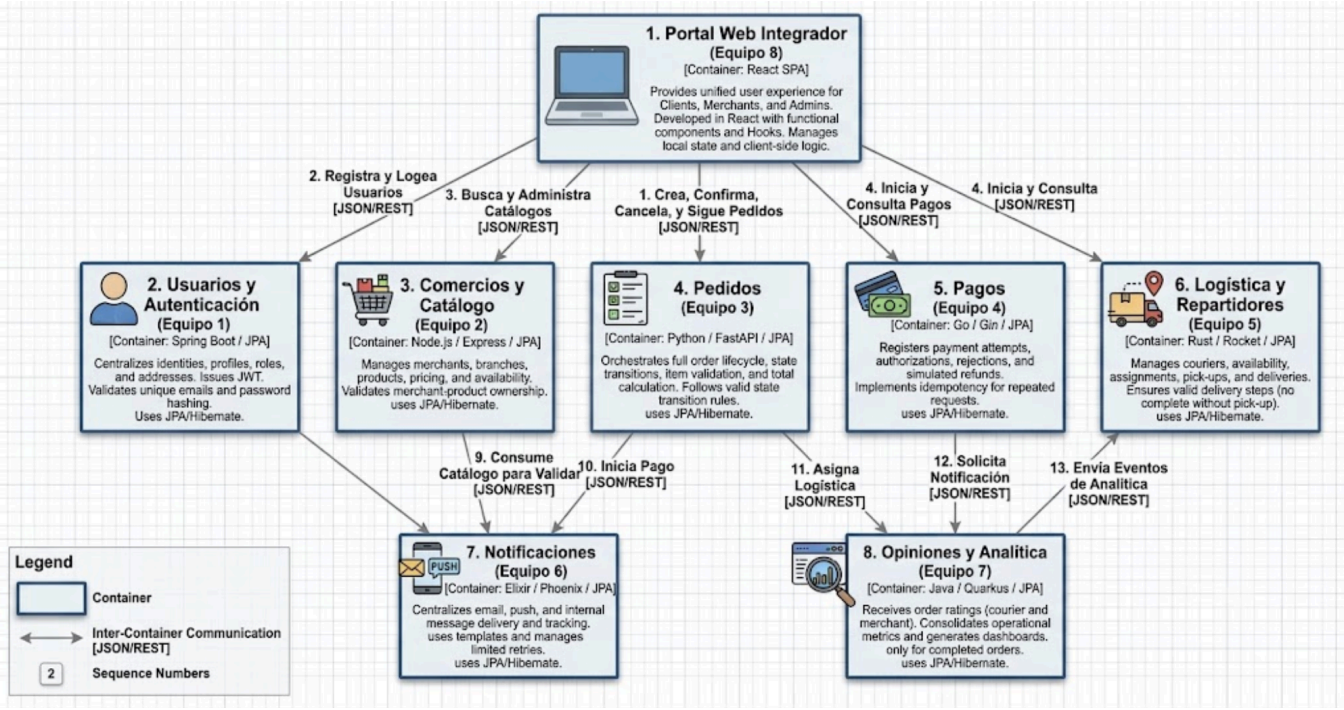


Figura 2. Servicios y relaciones principales.

Principios arquitectónicos

- Base de datos por módulo.
- Prohibido acceder directamente a tablas ajenas.
- Contratos OpenAPI versionados.
- DTO como formato de intercambio.
- Errores normalizados.
- Autenticación centralizada y autorización por rol.
- Integración sincrónica por REST; eventos/webhooks como ampliación.

4.1 Equipo 1 - Usuarios y Autenticación

Contexto y responsabilidad

Centraliza identidades, perfiles, roles y direcciones.

Entidades sugeridas

Usuario, Rol, Dirección, Sesión

Casos de uso mínimos

- Registrar usuario
- iniciar sesión
- modificar perfil
- administrar direcciones
- consultar usuario.

Reglas de negocio

- Email único.
- Contraseña nunca se almacena en texto plano.
- Usuario inactivo no puede iniciar sesión.
- JWT debe identificar usuario y roles.

APIs obligatorias

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- GET /api/v1/users/{id}
- PUT /api/v1/users/{id}
- POST /api/v1/users/{id}/addresses

Integraciones

- Notificaciones para recuperación o bienvenida.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.2 Equipo 2 - Comercios y Catálogo

Contexto y responsabilidad

Gestiona comercios, sucursales, categorías, productos, precios y disponibilidad.

Entidades sugeridas

Comercio, Sucursal, Categoría, Producto, Opción

Casos de uso mínimos

- Crear comercio
- publicar productos
- cambiar precio
- habilitar disponibilidad
- buscar catálogo.

Reglas de negocio

- Solo el comercio propietario puede modificar su catálogo.
- Producto no disponible no puede agregarse a un pedido.
- Precio vigente debe informarse en la consulta.

APIs obligatorias

- POST /api/v1/merchants
- POST /api/v1/merchants/{id}/products
- GET /api/v1/catalog
- GET /api/v1/products/{id}
- PATCH /api/v1/products/{id}/availability

Integraciones

- Usuarios para validar responsables; Notificaciones para cambios relevantes.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.3 Equipo 3 - Pedidos

Contexto y responsabilidad

Orquesta el ciclo del pedido y mantiene el estado funcional de la compra.

Entidades sugeridas

Pedido, ItemPedido, EstadoPedido, HistorialEstado

Casos de uso mínimos

- Crear pedido
- agregar ítems
- confirmar
- cancelar
- consultar estado
- listar pedidos.

Reglas de negocio

- Pedido debe pertenecer a un usuario válido.
- Todos los productos deben ser del mismo comercio.
- Total se calcula con precios confirmados.
- Estados siguen transición válida.

APIs obligatorias

- POST /api/v1/orders
- POST /api/v1/orders/{id}/items
- POST /api/v1/orders/{id}/confirm
- POST /api/v1/orders/{id}/cancel
- GET /api/v1/orders/{id}

Integraciones

- Usuarios, Catálogo, Pagos, Logística, Notificaciones y Analítica.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.4 Equipo 4 - Pagos

Contexto y responsabilidad

Registra intentos de pago, autorizaciones, rechazos y devoluciones simuladas.

Entidades sugeridas

Pago, MedioPago, EstadoPago, Reembolso

Casos de uso mínimos

- Iniciar pago
- autorizar/rechazar
- consultar
- reembolsar
- listar pagos.

Reglas de negocio

- Una solicitud repetida con misma clave idempotente no duplica pago.
- Importe debe coincidir con pedido.
- Solo pago aprobado permite avanzar el pedido.

APIs obligatorias

- POST /api/v1/payments
- GET /api/v1/payments/{id}
- POST /api/v1/payments/{id}/approve
- POST /api/v1/payments/{id}/reject
- POST /api/v1/payments/{id}/refund

Integraciones

- Pedidos, Usuarios y Notificaciones.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.5 Equipo 5 - Repartidores y Logística

Contexto y responsabilidad

Administra repartidores, disponibilidad, asignación, retiro y entrega.

Entidades sugeridas

Repartidor, Vehículo, Entrega, Zona, EstadoEntrega

Casos de uso mínimos

- Registrar repartidor
- cambiar disponibilidad
- asignar
- retirar
- entregar
- consultar seguimiento.

Reglas de negocio

- Solo repartidor disponible puede recibir asignación.
- Una entrega activa por repartidor.
- No puede entregarse sin retiro previo.

APIs obligatorias

- POST /api/v1/couriers
- PATCH /api/v1/couriers/{id}/availability
- POST /api/v1/deliveries/assign
- POST /api/v1/deliveries/{id}/pickup
- POST /api/v1/deliveries/{id}/complete

Integraciones

- Pedidos, Usuarios y Notificaciones.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.6 Equipo 6 - Notificaciones

Contexto y responsabilidad

Centraliza mensajes de email, push o bandeja interna y su trazabilidad.

Entidades sugeridas

Notificación, Plantilla, Canal, EstadoEnvio

Casos de uso mínimos

- Crear notificación
- usar plantilla
- reintentar
- consultar estado
- listar historial.

Reglas de negocio

- Cada solicitud debe tener destinatario, canal y plantilla válida.
- Reintentos limitados.
- No guardar contenido sensible innecesario.

APIs obligatorias

- POST /api/v1/notifications
- GET /api/v1/notifications/{id}
- POST /api/v1/notifications/{id}/retry
- GET /api/v1/users/{id}/notifications
- POST /api/v1/templates

Integraciones

- Usuarios para datos mínimos del destinatario; recibe solicitudes de todos.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.7 Equipo 7 - Opiniones y Analítica

Contexto y responsabilidad

Recibe calificaciones y consolida métricas operativas del ecosistema.

Entidades sugeridas

Reseña, Calificación, Métrica, EventoAnalítico

Casos de uso mínimos

- Calificar comercio
- calificar repartidor
- consultar promedio
- registrar evento
- generar tablero.

Reglas de negocio

- Solo pedidos entregados pueden calificarse.
- Una reseña por usuario y pedido.
- Métricas deben indicar período y fuente.

APIs obligatorias

- POST /api/v1/reviews
- GET /api/v1/merchants/{id}/rating
- GET /api/v1/couriers/{id}/rating
- POST /api/v1/analytics/events
- GET /api/v1/analytics/dashboard

Integraciones

- Pedidos, Usuarios, Pagos y Logística.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

4.8 Equipo 8 - Portal Web Integrador

Contexto y responsabilidad

Provee la experiencia web unificada para clientes y vistas mínimas administrativas.

Entidades sugeridas

Componentes UI, sesión, carrito local, vistas de seguimiento

Casos de uso mínimos

- Registrarse
- iniciar sesión
- explorar catálogo
- gestionar carrito
- comprar
- pagar
- seguir pedido
- calificar.

Reglas de negocio

- No implementa lógica de negocio propia.
- Debe consumir contratos publicados.
- Debe manejar fallos parciales y mensajes comprensibles.

APIs obligatorias

- Consume endpoints de los siete servicios.

Integraciones

- Usuarios, Catálogo, Pedidos, Pagos, Logística, Notificaciones y Analítica.

El equipo puede agregar funcionalidades, pero no eliminar los casos de uso ni contratos obligatorios sin aprobación docente.

5. Caso de uso transversal: Realizar un pedido

Precondiciones

- Usuario registrado y autenticado.
- Comercio activo.
- Productos disponibles.
- Dirección válida.
- Servicios accesibles o respuestas simuladas acordadas.

Flujo principal

8. El cliente inicia sesión.
9. Explora comercios y productos.
10. Agrega productos al carrito.
11. El Portal solicita la creación del pedido.
12. Pedidos valida usuario, catálogo y precios.
13. Pagos procesa la autorización simulada.
14. Pedidos confirma la operación.
15. Logística asigna un repartidor.
16. Notificaciones informa la confirmación.
17. El cliente consulta el seguimiento hasta la entrega.

Alternativas

- Producto sin disponibilidad.
- Precio modificado.
- Pago rechazado.
- Sin repartidores disponibles.
- Cancelación dentro de estado permitido.
- Servicio externo temporalmente no disponible.

5. Caso de uso transversal: Realizar un pedido

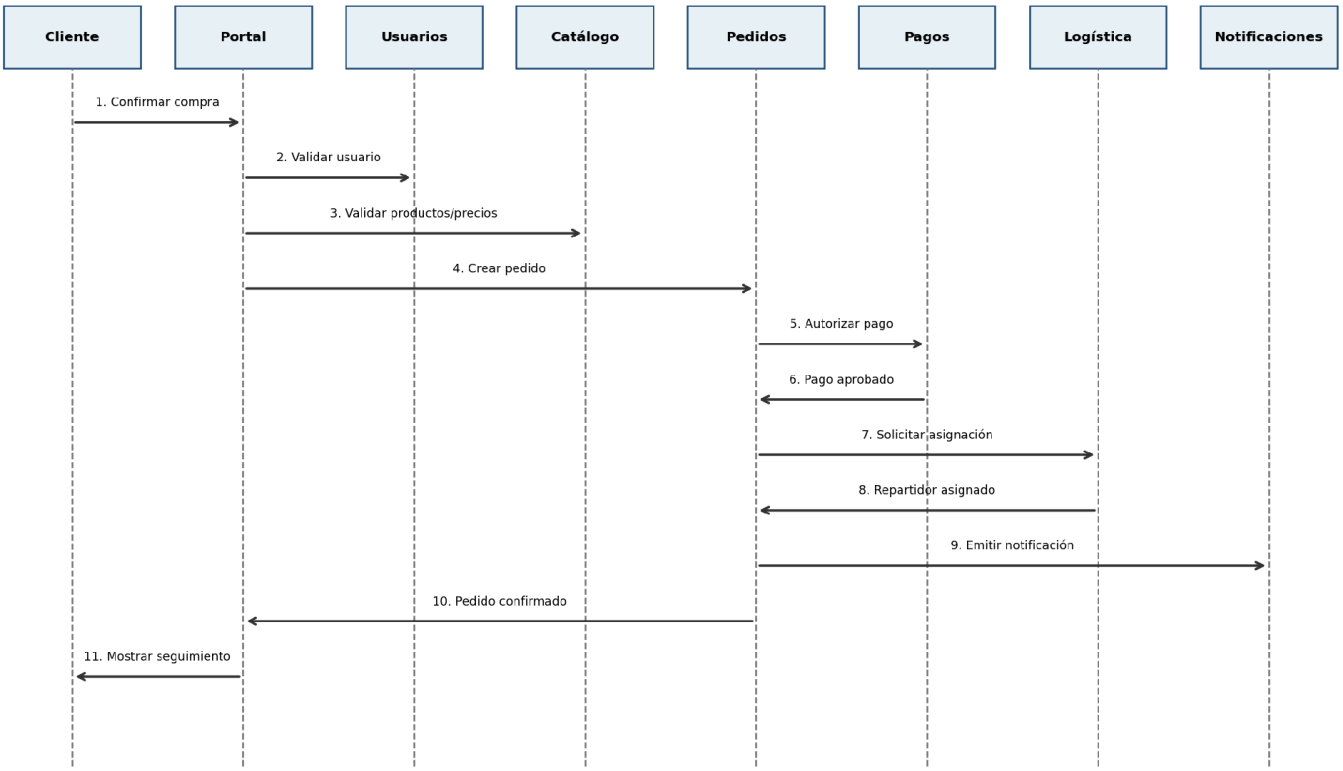


Figura 3. Diagrama de secuencia simplificado del flujo principal.

Estados sugeridos

Pedido	Pago	Entrega
BORRADOR	PENDIENTE	-
CONFIRMADO	APROBADO	PENDIENTE_ASIGNACION
EN_PREPARACION	APROBADO	ASIGNADA
EN_CAMINO	APROBADO	RETIRADA
ENTREGADO	APROBADO	COMPLETADA
CANCELADO	RECHAZADO/REEMBOLSADO	CANCELADA

6. Contratos de integración y APIs REST

Convenciones obligatorias

Elemento	Convención
Base URL	/api/v1
Formato	application/json; UTF-8
Identificadores	UUID o Long, consistente por módulo
Fechas	ISO-8601
Paginación	page, size, sort
Correlación	X-Correlation-Id recomendado
Autenticación	Bearer JWT
Versionado	Cambios incompatibles requieren nueva versión

Códigos HTTP

Código	Uso
200	Consulta o actualización exitosa
201	Recurso creado
204	Operación sin cuerpo
400	Solicitud inválida
401	Sin autenticación
403	Sin autorización
404	Recurso inexistente
409	Conflicto de estado o duplicado
422	Regla de negocio incumplida
500	Error no controlado

6. Contratos de integración y APIs REST

Formato de error normalizado

```
{
  "timestamp": "2026-08-11T20:15:30Z",
  "status": 422,
  "code": "ORDER_INVALID_STATE",
  "message": "El pedido no puede confirmarse en su estado actual",
  "path": "/api/v1/orders/123/confirm",
  "correlationId": "..."
}
```

Ejemplo de creación de pedido

POST /api/v1/orders

```
{
  "userId": "...",
  "merchantId": "...",
  "deliveryAddressId": "...",
  "items": [{"productId":"...","quantity":2}]
}
```

Respuesta

```
{
  "id": "...",
  "status": "DRAFT",
  "total": 18500.00,
  "currency": "ARS"
}
```

6. Contratos de integración y APIs REST

Obligatorias y opcionales

Categoría	Requisito
Obligatoria	Endpoints mínimos definidos para cada módulo.
Obligatoria	Swagger UI accesible durante la demo.
Obligatoria	Prueba de al menos tres integraciones reales.
Opcional	Webhooks para notificaciones.
Opcional	Eventos asíncronos con broker.
Opcional	Cache, rate limiting o circuit breaker.
Opcional	Observabilidad con métricas y trazas.

Reglas de dependencia

- Ningún servicio lee la base de datos de otro.
- Los DTO externos no se reutilizan como entidades JPA.
- Los equipos acuerdan contratos antes de implementar.
- Todo cambio se comunica y versiona.
- Durante el desarrollo pueden utilizarse mocks temporales.

7. Organización de equipos, Git y estándares

Distribución

Equipo	Módulo	Integrantes
1	Usuarios y autenticación	5
2	Comercios y catálogo	4
3	Pedidos	4
4	Pagos	4
5	Logística	4
6	Notificaciones	4
7	Opiniones y analítica	4
8	Portal integrador	4

Roles sugeridos

- Referente de integración.
- Responsable backend.
- Responsable frontend.
- Responsable de calidad/documentación.
- Responsabilidades rotativas para evitar especialización rígida.

Todos los integrantes deben comprender el módulo completo. La defensa individual puede abordar cualquier parte de la solución.

7. Organización de equipos, Git y estándares

Estrategia Git

Rama	Uso
main	Versiones estables y entregas.
develop	Integración del equipo.
feature/<tema>	Nueva funcionalidad.
fix/<tema>	Corrección.
release/<versión>	Preparación de entrega, opcional.

Estándares

- Pull Request obligatorio hacia develop/main.
- Commits descriptivos.
- README actualizado.
- Variables sensibles fuera del repositorio.
- Convenciones uniformes de nombres.
- Separación controller/service/repository.
- DTO, validaciones y manejo centralizado de errores.
- Revisión cruzada de código.

Estructura sugerida

src/main/java/...
controller/
service/
repository/
domain/
dto/
mapper/
exception/
config/
src/test/...
docs/
README.md
docker-compose.yml

8. Entregables y cronograma

Etapa	Contenido mínimo	Resultado
Entrega 1 - Análisis y arquitectura	Alcance, actores, reglas, C4, UML, DER, backlog, repositorio.	Diseño aprobado antes del desarrollo.
Entrega 2 - Backend y persistencia	Entidades, repositorios, servicios, controladores, validaciones, Swagger y pruebas.	API funcional y documentada.
Entrega 3 - Cliente e integración parcial	Interfaz, consumo de APIs, manejo de sesión/errores, tres integraciones.	Flujo parcial demostrable.
Entrega final	Sistema completo, integraciones, documentación, pruebas y demo.	Ecosistema ejecutable de punta a punta.

Documentación por entrega

- README con instrucciones.
- Acta de decisiones técnicas.
- Diagramas actualizados.
- Colección de pruebas o evidencia.
- Registro de integraciones y responsables.
- Backlog con estado de tareas.

8. Entregables y cronograma

Criterios de aceptación comunes

- Compila y ejecuta desde un entorno limpio.
- No requiere credenciales reales.
- Datos de prueba reproducibles.
- Endpoints documentados y verificables.
- No existen accesos directos a bases ajenas.
- Errores controlados y mensajes claros.
- Integraciones evidenciadas.
- Participación comprobable en Git.

Política de cambios

Los equipos pueden proponer mejoras o cambios de alcance. Deben registrar la decisión, analizar impacto sobre otros módulos y obtener aprobación docente antes de modificar un contrato obligatorio.

Seguimiento semanal

Control	Evidencia
Avance	Tablero y commits.
Riesgos	Registro breve y responsable.
Integración	Prueba o mock acordado.
Calidad	Issues, revisión o test.
Documentación	Diagramas y README actualizados.

9. Rúbricas de evaluación y Demo Day

Criterio	Peso	Indicadores
Arquitectura y modelado	15%	Límites, C4/UML, modelo y decisiones.
Backend y persistencia	20%	Capas, reglas, ORM, validaciones y errores.
Frontend/experiencia	15%	Navegación, interacción, accesibilidad básica y consumo.
Integración	20%	Contratos, interoperabilidad, resiliencia y flujo transversal.
Documentación	10%	README, OpenAPI, diagramas y evidencias.
Calidad y Git	10%	Legibilidad, pruebas, ramas, PR y trazabilidad.
Defensa individual	10%	Comprensión, argumentación y participación.

Escala

Nivel	Descripción
Excelente	Completo, consistente, integrado y defendido con fundamentos.
Bueno	Cumple lo esencial con errores menores.
Básico	Funciona parcialmente o presenta debilidades de diseño.
Insuficiente	No cumple el flujo, contratos o evidencia de autoría.

9. Rúbricas de evaluación y Demo Day

Demo Day

18. Preparar entorno y datos antes del turno.
19. Presentar contexto y responsabilidad del módulo.
20. Demostrar el caso principal.
21. Mostrar al menos tres integraciones.
22. Evidenciar manejo de un error.
23. Explicar arquitectura y contratos.
24. Responder preguntas grupales e individuales.

Duración orientativa

Actividad	Tiempo
Presentación del módulo	5 min
Demostración funcional	10 min
Integración transversal	5 min
Preguntas y defensa	10 min

La presentación no reemplaza el funcionamiento. Las diapositivas deben apoyar la explicación; la evaluación principal se realiza sobre el sistema, la documentación y la defensa.

Condiciones críticas

- La falta de participación individual puede modificar la nota del integrante.
- Una integración simulada debe estar declarada; no puede presentarse como real.
- El plagio o la autoría no demostrable se rige por las normas académicas.

10. Anexos

Checklist técnico

- Proyecto ejecuta localmente.
- Configuración documentada.
- Base de datos inicializable.
- Swagger disponible.
- Estados y reglas implementados.
- Errores normalizados.
- Datos sensibles excluidos.
- Pruebas o evidencias presentes.
- Docker opcional funcionando si se incluye.

Checklist de integración

- Contrato aprobado.
- URL y versión acordadas.
- DTO documentado.
- Timeout y errores contemplados.
- Prueba exitosa registrada.
- Responsables identificados.
- Fallback o mensaje al usuario definido.

Checklist documental

- C4 actualizado.
- UML de clases.
- Secuencia principal.
- DER.
- OpenAPI.
- README.
- Decisiones técnicas.
- Instrucciones para demo.

10. Anexos

Modelo UML de referencia

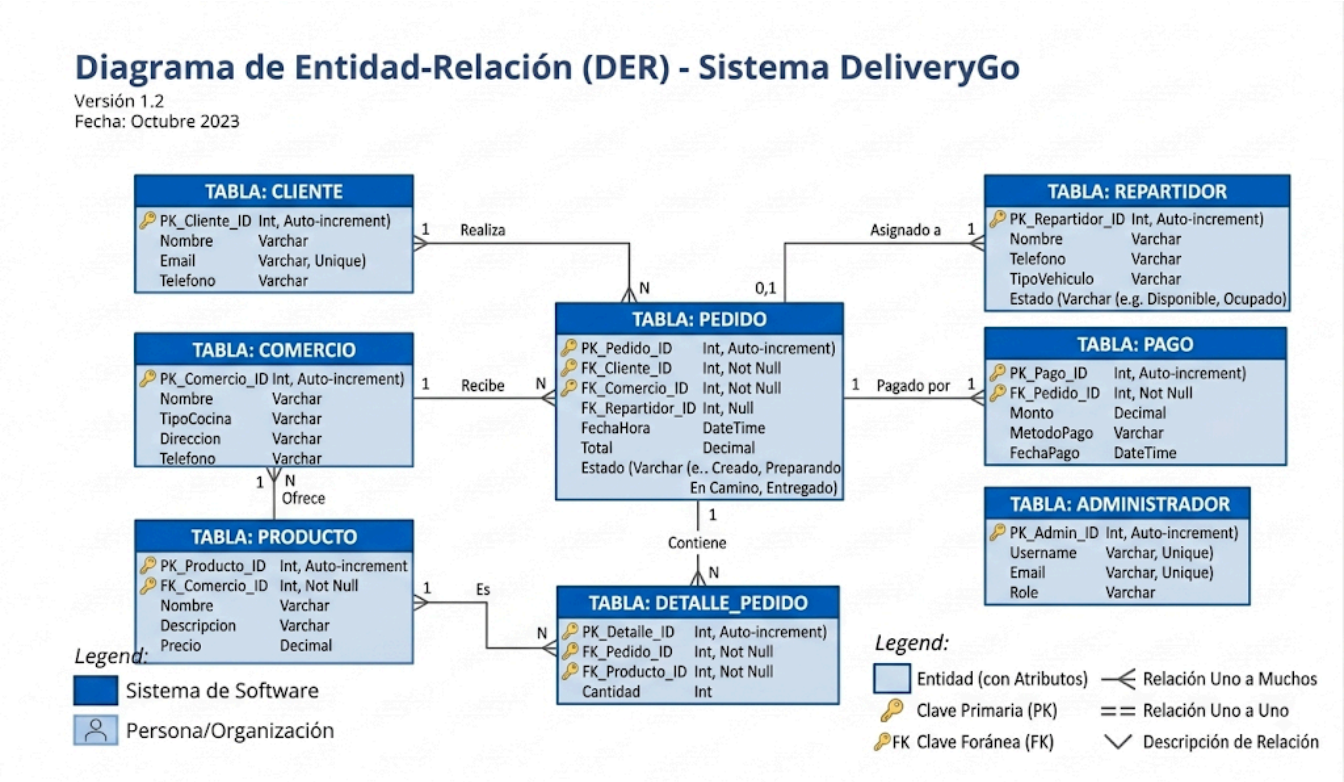


Figura 4. Modelo conceptual simplificado; cada equipo debe especializar su propio modelo.

Mockup orientativo

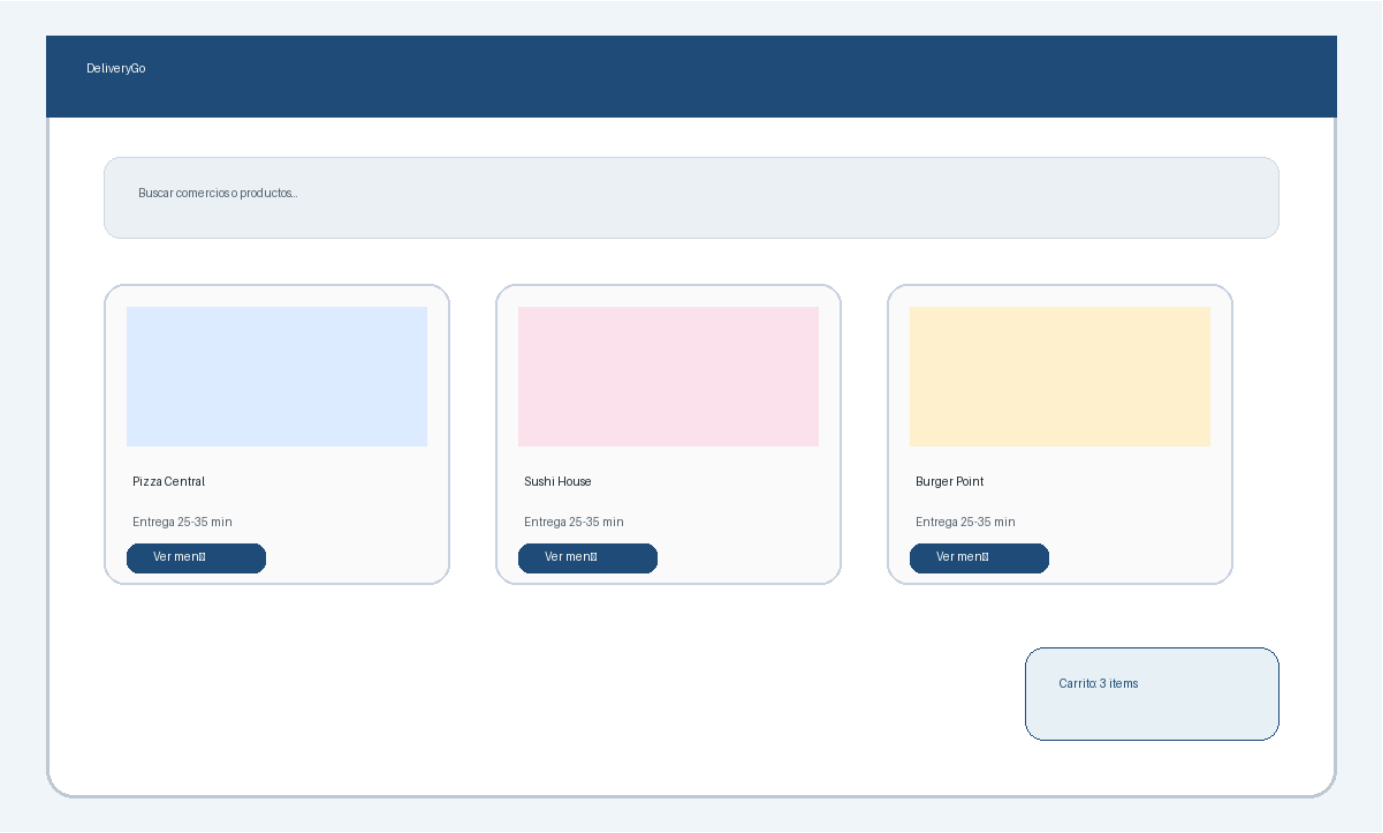


Figura 5. Referencia visual no vinculante para el Portal Web.

Contenido mínimo del README

- Descripción del módulo.
- Integrantes y responsabilidades.
- Tecnologías.
- Requisitos.
- Instalación y ejecución.
- Variables de entorno.
- Endpoints y Swagger.
- Integraciones.
- Datos de prueba.
- Limitaciones conocidas.

Referencias académicas y técnicas

El proyecto se alinea con el Programa Analítico de Aplicaciones Interactivas 3.4.082: persistencia, arquitectura backend, aplicaciones web, frameworks cliente/servidor, accesibilidad, usabilidad, protocolos y formatos de intercambio. Los detalles tecnológicos se complementarán con la documentación oficial utilizada durante la cursada.

Aprobación y uso del documento

Este documento define el alcance base del TPO. Las aclaraciones publicadas por el equipo docente durante el cuatrimestre se considerarán anexos válidos. Ante una ambigüedad, debe consultarse antes de implementar una decisión que afecte a otros equipos.